PRACTICAL NO 4 : GESTURE RECOGNITION FOR HUMAN-COMPUTER INTERACTION

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()

    if not ret:
        break

    frame = cv2.flip(frame, 1)

    # region of interest
    roi = frame[100:400, 300:600]

    # Draw ROI rectangle
    cv2.rectangle(frame, (300,100), (600,400), (0,255,0), 2)

    # convert to hsv
    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

    # skin color range
    lower_skin = np.array([0,20,70], dtype=np.uint8)
    upper_skin = np.array([20,255,255], dtype=np.uint8)

    mask = cv2.inRange(hsv, lower_skin, upper_skin)

    # remove noise
    kernel = np.ones((5,5), np.uint8)
```

```
    mask = cv2.GaussianBlur(mask, (5,5), 0)
    mask = cv2.morphologyEx(
        mask,
        cv2.MORPH_OPEN,
        kernel
    )

    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    gesture = "NO HAND"

    if contours:
        # find largest contour
        contour = max(contours, key=cv2.contourArea)

        area = cv2.contourArea(contour)

        if area > 3000:
            # draw hand contour
            cv2.drawContours(
                roi,
                [contour],
                -1,
                (255,0,0),
                2
            )
```

```
        # convex hull
        hull = cv2.convexHull(
            contour,
            returnPoints=False
        )

        if hull is not None and len(hull) > 3:
            defects = cv2.convexityDefects(
                contour,
                hull
            )

            finger_count = 0

            if defects is not None:
                for i in range(defects.shape[0]):
                    s,e,f,d = defects[i].ravel()

                    start = contour[s][0]
                    end = contour[e][0]
                    far = contour[f][0]

                    # triangle side lengths
                    a = np.linalg.norm(end - start)
                    b = np.linalg.norm(far - start)
                    c = np.linalg.norm(end - far)

                    if b != 0 and c != 0:
                        angle = np.arccos(
                            (b**2 + c**2 - a**2) /
                            (2*b*c)
                        )
```

```
                angle = angle * 180 / np.pi

                if angle <= 90:
                    finger_count += 1

                    cv2.circle(
                        roi,
                        tuple(far),
                        5,
                        (0,0,255),
                        -1
                    )

            # gesture recognition
            if finger_count == 0:
                gesture = "FIRST /ONE"

            elif finger_count == 1:
                gesture = "TWO / VICTORY"

            elif finger_count == 2:
                gesture = "THREE"

            elif finger_count == 3:
                gesture = "FOUR"

            elif finger_count >= 4:
                gesture = "OPEN PALM"

        # display detected gesture
        cv2.putText(
```

```
        frame,
        "Gesture: " + gesture,
        (20,60),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        (0,255,0),
        2
    )

    cv2.putText(
        frame,
        "PLace hand inside green box",
        (300,80),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0,255,0),
        2
    )

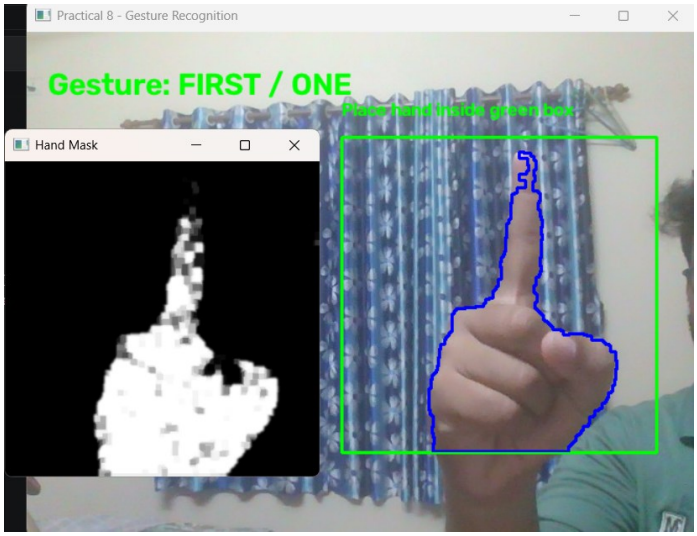
    cv2.imshow("Practical 8 - Gesture Recognition", frame)
    cv2.imshow("Hand Mask", mask)

    # press Q to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

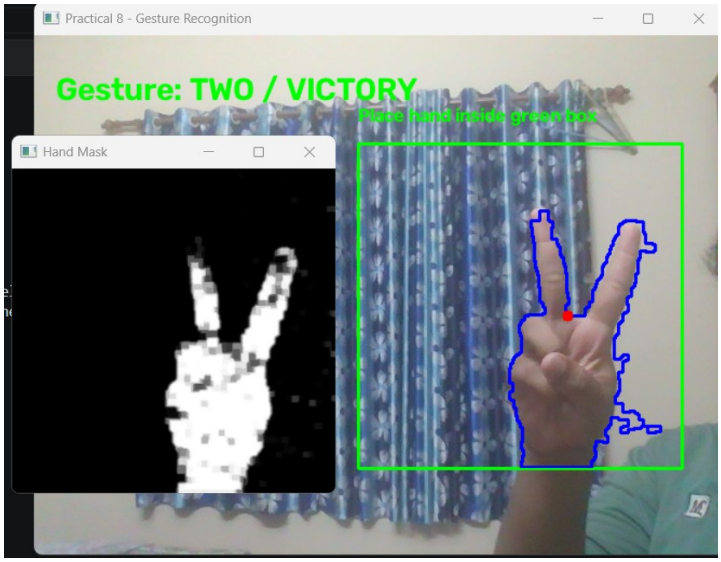
cap.release()
cv2.destroyAllWindows()
```

Output :

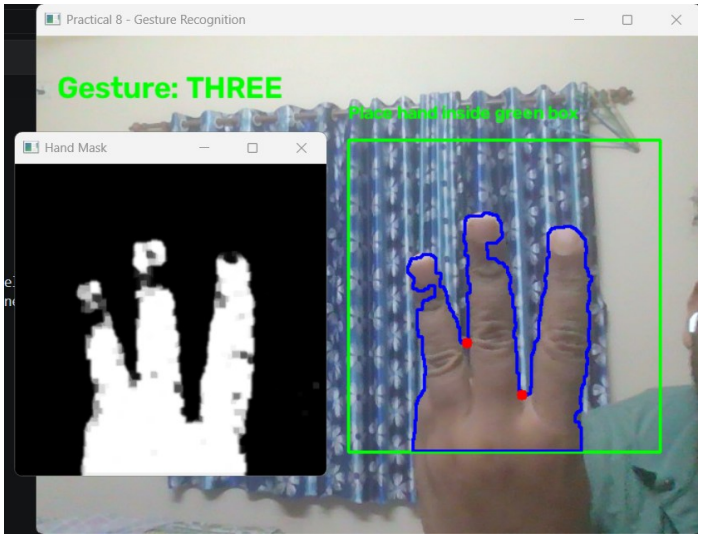
First/One finger ;



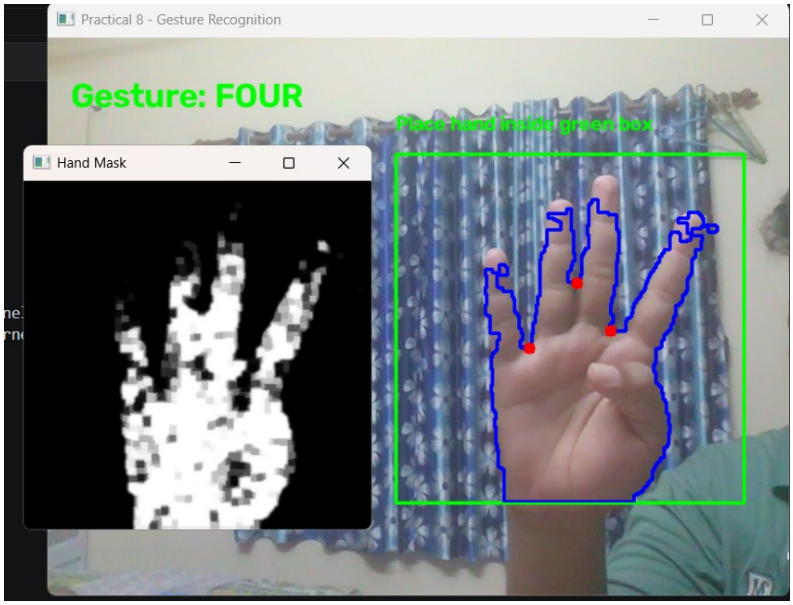
Two fingers / victory :



Three :



Four :



Open palm:

